



'DOM Scraping' and Cross Domain Mashups with Aptana Jaxer

One of the things I was working on recently is 'screen scraping' some data from several sites and presenting them in a single page. Of course Jaxer seems like the perfect tool for this as I can go fetch the data server-side, pull it all together, then present the final page to the browser. It was so exciting to see this work that I thought I'd make this post to share my experience with you!

We're working now on adding support in Jaxer for being able to programmatically create 'window' objects, filling them with content from a remote URL, having that content actually execute, then being able to go into that window object and pull DOM elements out. (Update: see [the Jaxer 1.0 version.](#))

Unfortunately for me, that work isn't ready yet, so I wanted to see if there was some workaround I could use until it was ready. Well, after some research and trial and error, a workaround was found. The only catch with this code is that if there is any JavaScript on the page(s) you are fetching, that code will not execute, that's coming soon.

Note that all of the code below can be contained within a single script tag:

```
1. <script runat='server'>...all code here...</script>
```

So, the first thing I wanted to do is make it really easy to fetch a remote document and grab out some content. I didn't want to use string matching or regular expressions of any kind, I wanted to use my trusted 'getElementById' to fetch named elements directly. (Of course I ran into sites that don't name their elements, but fortunately many use named classes, so I had to fetch them that way, but I get ahead of myself.)

Let's get started. The following two lines is all that is required to fetch a remote URL synchronously on the server and then pull out a named element:

```
1. var doc = getDocumentFromURL("http://www.news.com/");
2. var item = doc.getElementById("mostPopStories");
```

Now that we have 'item' as the element we were trying to fetch, we just have to add that element to our current blank page. I created a simple function called 'addElementToPage(title, element)' that takes care of this for me. Here's the next line:

```
1. addElementToPage("News.com Popular Stories", item);
```

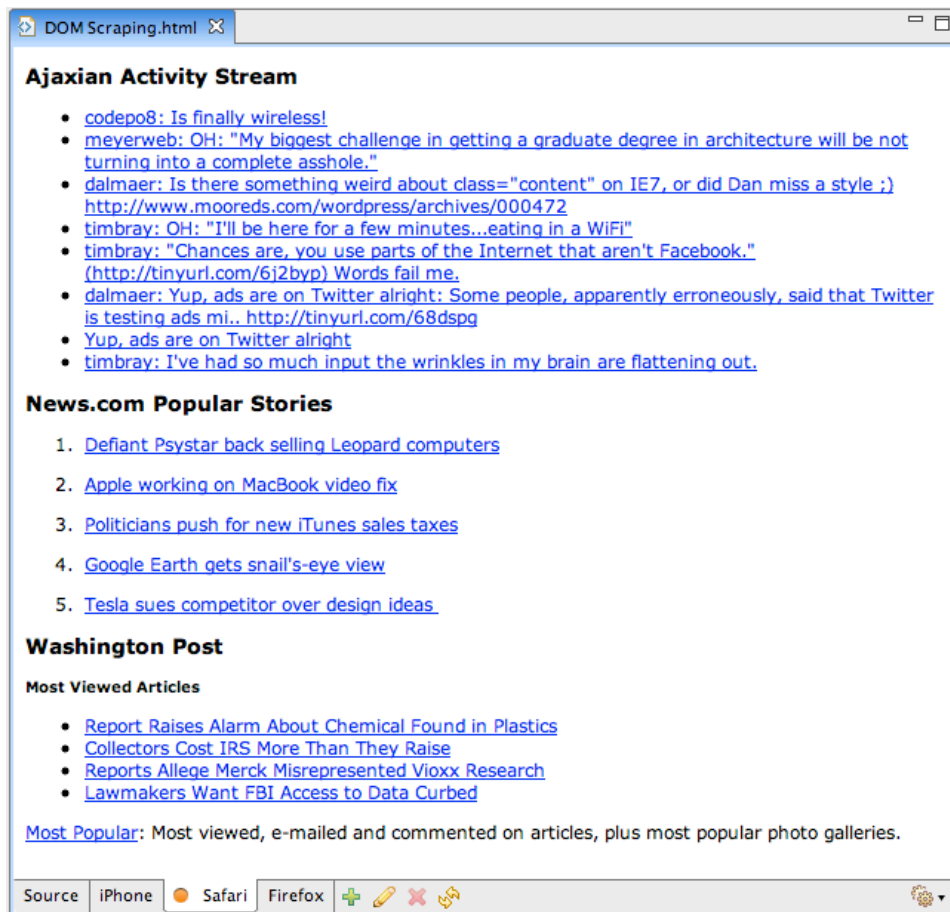
That's it. Now I do that 2 more times to fetch content from 2 other sites.

```
1. doc = getDocumentFromURL("http://ajaxian.com");
2. item = getElementsByClass('activitystream', doc)[0];
3. addElementToPage("Ajaxian Activity Stream", item);
4.
5. doc = getDocumentFromURL("http://www.washingtonpost.com/");
6. item = getElementsByClass("top-box-in", doc)[0];
7. addElementToPage("Washington Post", item);
```

You'll notice that I didn't use getElementById() as those elements were not named, but fortunately they had class names associated with them. I found a convenient function called getElementsByClass() that I used in this case. Since it returns an array of elements, I use the [0] index to retrieve the first item in the list, which in my case, is probably the only item.

Believe it or not, that's basically it. At this point, your three 'DOM fragments' have been fetched and inserted into your new document. Here's the result:





Following are the convenience functions that help make the magic happen. The first function is `addElementToPage()`. It very simply just creates an H2 tag and sets the title and a DIV tag which contains the contents of the fetched element:

```
01. function addElementToPage(title, element)
02. {
03.     var el = document.importNode(element, true);
04.
05.     var h2 = document.createElement('h2');
06.     h2.innerHTML = title;
07.     document.body.appendChild(h2);
08.
09.     var div = document.createElement('div');
10.     div.appendChild(el);
11.
12.     document.body.appendChild(div);
13. }
```

This next function, `getDocumentFromURL()`, is the one that does most of the work. (I found some good info on the subject of HTML to DOM here:

<http://jszen.blogspot.com/2007/02/how-to-parse-html-strings-into-dom.html>) It first goes and retrieves the remote page. Then it creates a 'document fragment' from the contents of the fetched site. That fragment is then added to a dynamically created IFRAME. Finally, the Document object from the IFRAME is fetched and returned. In short, we can pass in a URL, get the string value, place it into an IFRAME, then pull out the resulting Document object so that we can work on it.

```
01. function getDocumentFromURL(url)
02. {
03.     // Get remote site
04.     var string = Jaxer.Web.get(url);
05.
06.     // Parse the string into a doc fragment
07.     var range = document.createRange();
08.     range.selectNode(document.body);
09.     var parsedHTML = range.createContextualFragment(string);
10.
11.     // Create the iframe that will contain the site we loaded
12.     var iframe = document.createElement("iframe");
13.     document.body.appendChild(iframe);
14.
15.     // Import from doc fragment into the iframe
16.     var el = iframe.contentDocument.importNode(parsedHTML, true);
17.     iframe.contentDocument.body.appendChild(el);
18.
19.     var newDoc = iframe.contentDocument;
20.
21.     // delete the iframe as we don't need it any more
22.     document.body.removeChild(iframe);
23.
24.     return newDoc;
25. }
```

This final function was found at <http://www.dustindiaz.com/getelementsbyclass/> and walks a node looking for an element with a specific class name. It was used in the case where elements don't have ID's, so a class name is used instead.

```
01. function getElementsByClass(searchClass, node, tag)
02. {
03.     var classElements = new Array();
04.     if (node == null)
05.         node = document;
06.     if (tag == null)
```

```
07.         tag = '*';
08.         var els = node.getElementsByTagName(tag);
09.         var elsLen = els.length;
10.         var pattern = new RegExp("(^|\\\\s)" + searchClass + "(\\\\s|$)");
11.         for (i = 0, j = 0; i < elsLen; i++) {
12.             if (pattern.test(els[i].className)) {
13.                 classElements[j] = els[i];
14.                 j++;
15.             }
16.         }
17.         return classElements;
18.     }
```

What's exciting about this sample is that it is relatively simple, uses the full power of server-side JavaScript and more importantly, Jaxer's cool server-side DOM capability to enable real 'DOM scraping'. Once window object creation is finished in Jaxer (real soon now), then you'll be able to fetch remote pages, execute their integrated code, then proceed to fetch out items from the resulting DOM.

Update: see [the Jaxer 1.0 version](#).